

Mesh Generation & Optimization Algorithms

Assignment

1. Case Study 1

1.1. Grid Generation

For the assignment a random domain is defined. To further test the grid generation algorithm the same domain is used with added random noise on the boundaries.

The code used to generate the internal nodes of the grid is the following:

```
/// <summary>
/// This function is the grid-generation algorithm, more or less the pseudocode
/// of second course, written in regular C# code.
/// </summary>
public static Vec2[,] MeshFromBoundaryNodes (Sides sides, int Imax, int Jmax)
{
    Vec2[,] mesh = new Vec2[Imax,Jmax];

    // copy boundary nodes
    for (int i = 0; i < Imax; i++) {
        mesh[i,0] = sides.L[i];
        mesh[i,Jmax-1] = sides.R[i];
    }
    for (int j = 0; j < Jmax; j++) {
        mesh[0,j] = sides.D[j];
        mesh[Imax-1,j] = sides.U[j];
    }

    // generate internal nodes
    for (int i = 1; i < Imax - 1; i++) {
        Vec2 dξ = (sides.R[i] - sides.L[i]) / (double)(Jmax-1);

        for (int j = 1; j < Jmax - 1; j++) {
            Vec2 dη = (sides.U[j] - sides.D[j]) / (double)(Imax-1);

            mesh[i,j].X = mesh[i,j-1].X + dξ.X;
            mesh[i,j].Y = mesh[i-1,j].Y + dη.Y;
        }
    }

    return mesh;
}
```

In order to plot the grid with Gnuplot the vertices are stored as an ASCII file with appropriate format. The grids generated are the following four cases. One with straight lines and added noise and a second case with curved sides and random noise. The generated grids are for $N \times N$ with $N = 30$.

1

¹The code for the 1st Case Study is the `geom_randomizer.cs` and the `mesh_4_test.fsx`

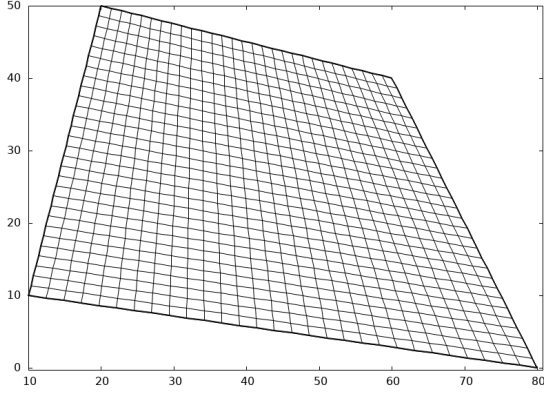


Figure 1: The generated grid from the above code snippet for a random domain

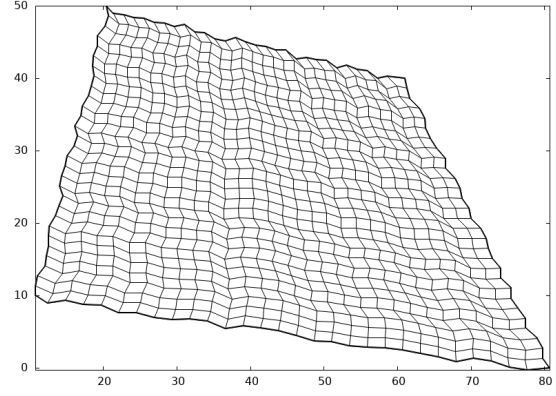


Figure 2: The generated grid with random noise on the boundaries

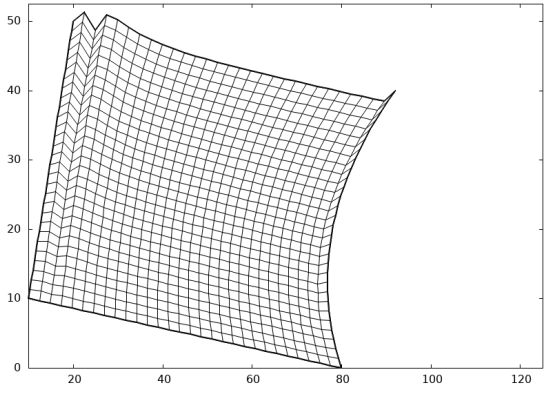


Figure 3: The generated grid from the above code snippet for a random domain with curved sides

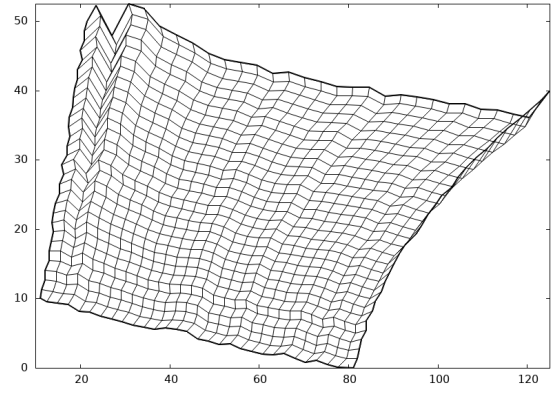


Figure 4: The generated grid with curved sides and random noise on the boundaries

1.2. Optimization

For the optimization part of the assignment the following function was used for measuring the intensity of the signal from the antennas.

$$f_{\text{signal}}(x, y) = A \cdot \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_x^2} + \frac{(y - y_0)^2}{2\sigma_y^2}\right)\right)$$

A : amplitude

x : x-coordinate of grid node

x_o : x-coordinate of the antenna

y : y-coordinate of grid node

y_o : y-coordinate of the antenna

σ_x : variance on x

σ_y : variance on y

Steepest Descend was utilized for the optimization of the antennas positions.

$$b^{n+1} = b^n - \eta \frac{\partial F}{\partial u}$$

The following code computes the gradient on each node:

```

/// <summary>
/// This function is for the second part of the assignment,
/// the optimization of the Intensity of the signal
/// </summary>
public static Gradient GetGradient (Vec2[,] mesh, double[,] signal, int i, int j)
{
    Vec2 p = mesh[i,j];
    double s = signal[i,j];

    return new Gradient{
        LL = (s - signal[i-1,j-1]) / (p - mesh[i-1,j-1]).Length(),
        LC = (s - signal[i-1,j+0]) / (p - mesh[i-1,j+0]).Length(),
        LR = (s - signal[i-1,j+1]) / (p - mesh[i-1,j+1]).Length(),

        UL = (s - signal[i+1,j-1]) / (p - mesh[i+1,j-1]).Length(),
        UC = (s - signal[i+1,j+0]) / (p - mesh[i+1,j+0]).Length(),
        UR = (s - signal[i+1,j+1]) / (p - mesh[i+1,j+1]).Length(),

        CL = (s - signal[i,j-1]) / (p - mesh[i,j-1]).Length(),
        CR = (s - signal[i,j+1]) / (p - mesh[i,j+1]).Length(),
    };
}

```

UL	UC	UR
CL	node	CR
LL	LC	LR

Table 1: The Gradient struct has the above layout. The fields of the struct are the neighboring nodes of the current node as displayed on the image

For the optimization loop the following predicate holds:

$$x_i - 0.7 \cdot \bar{x} > 0 \rightarrow \text{Gradient Descent}$$

$$x_i - 0.2 \cdot \bar{x} < 0 \rightarrow \text{Gradient Ascend}$$

1.3. Constraints

The following constraints are applied to prevent the antennas coincide or get outside of the domain.

$$(x_i - x_j)^2 - R^2 \geq 0$$

$$x_i - \text{sides}.L[i] \geq 0$$

$$\text{sides}.R[i] - x_i \geq 0$$

$$x_i - \text{sides}.D[i] \geq 0$$

$$\text{sides}.U[i] - x_i \geq 0$$

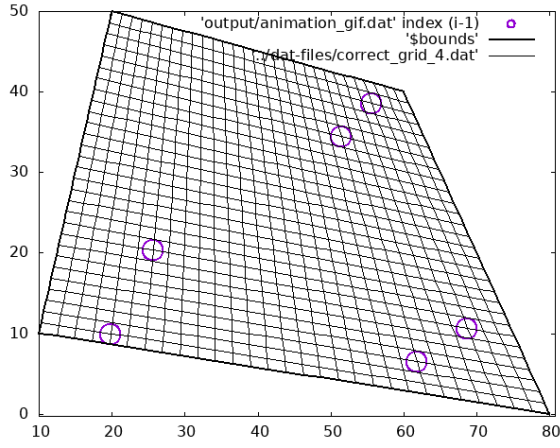


Figure 5: Gradient Descend Optimization with Constraints (Gif)

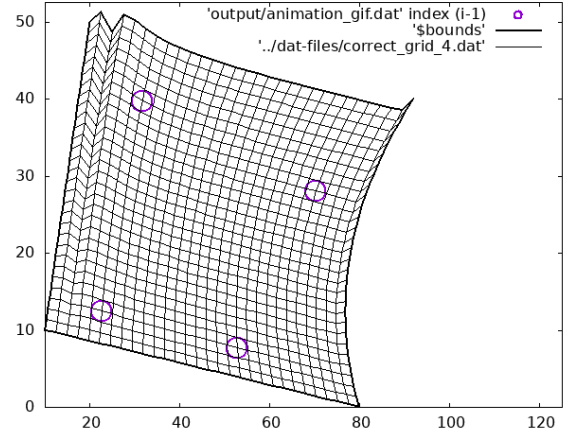


Figure 6: Gradient Descend Optimization with Constraints (Gif)

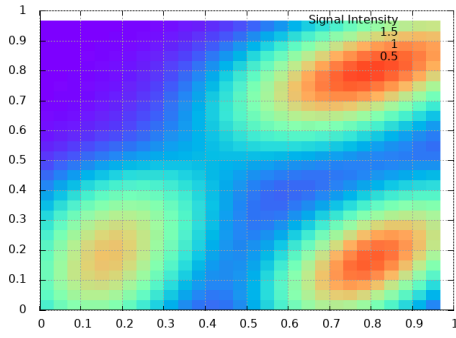


Figure 7: computational domain, straight-lines sides case

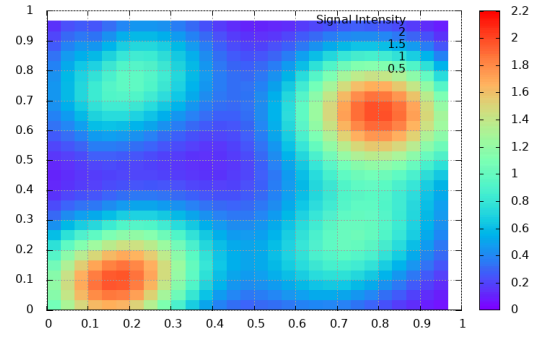


Figure 8: computational domain, curved-lines sides case

Antennas Positions	
X	Y
25.660	20.345
51.391	34.483
68.680	10.690
61.784	6.552
20.059	11.379
59.715	35.172

Antennas Positions	
X	Y
61.438	11.635
70.301	28.049
31.656	39.807
22.627	12.500
70.301	28.049
22.627	12.500

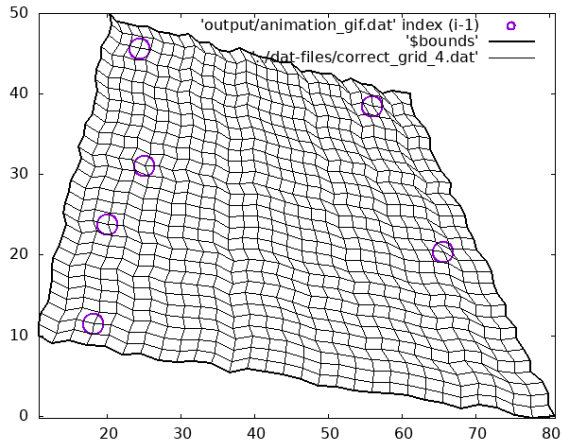


Figure 9: Gradient Descent Optimization with Constraints (Gif)

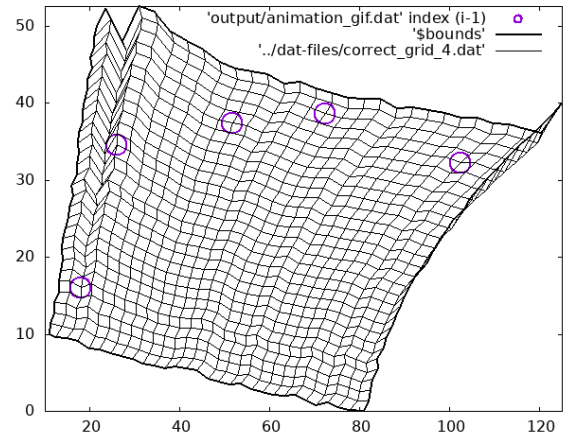


Figure 10: Gradient Descent Optimization with Constraints (Gif)

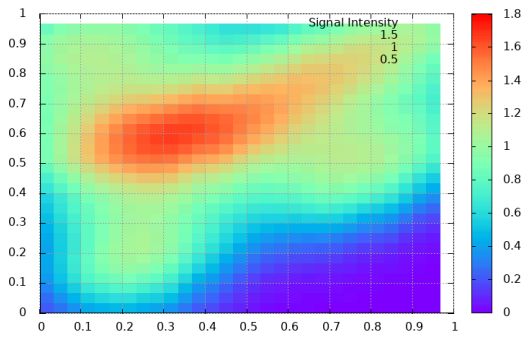


Figure 11: computational domain, straight-lines sides case (Gif)

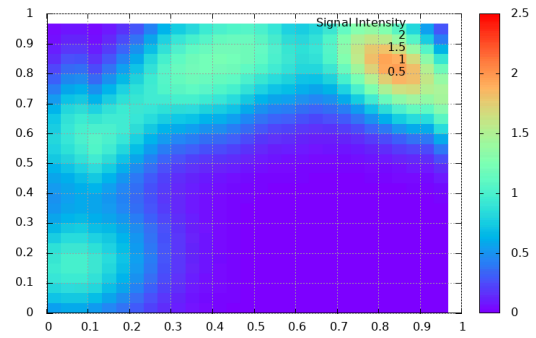


Figure 12: computational domain, curved-lines sides case (Gif)

Antennas Positions	
X	Y
25.188	31.119
24.506	45.645
60.730	22.075
27.979	15.252
55.977	38.473
40.101	30.384

Antennas Positions	
X	Y
51.717	37.467
26.108	34.566
17.967	16.041
102.423	32.224
102.423	32.224
72.460	38.707

Depending on the case whether the intensity of the node is above the maximum threshold or below the minimum then Gradient Descend or Gradient Ascend is applied respectively.

The code for the optimization loop is defined as:

```

for i in 1..N-2 do
  for j in 1..N-2 do
    let p = mesh[i,j]
    let k = get_closest_antenna p antennas_pos
    if signal[i,j] > (s * 0.7) then
      let I,J =
        match (GeoRandomizer.GetGradient(mesh, signal,
i,j).SteepestDescend()) with
          | GradDirection.LL -> (i-1,j-1)
          | GradDirection.LC -> (i-1,j+0)
          | GradDirection.LR -> (i-1,j+1)
          | GradDirection.UL -> (i+1,j-1)
          | GradDirection.UC -> (i+1,j+0)
          | GradDirection.UR -> (i+1,j+1)
          | GradDirection.CL -> (i,j-1)
          | GradDirection.CR -> (i,j+1)
          | _ -> (i,j)
      let (_i, _j) = (Math.Clamp(I, 1, N-2), Math.Clamp(J,1,N-2))
      let _p = mesh[_i, _j]
      if distance_constraint (I,J) antennas_idx && sides_constraint _p sides N
then
        antennas_idx[k] <- (_i,_j)
        antennas_pos[k] <- mesh[fst antennas_idx[k], snd antennas_idx[k]]

      if signal[i,j] < (s * 0.2) then
        let I,J =
          match (GeoRandomizer.GetGradient(mesh, signal, i,j).SteepestAscend())
with
          | GradDirection.LL -> (i-1,j-1)
          | GradDirection.LC -> (i-1,j+0)
          | GradDirection.LR -> (i-1,j+1)
          | GradDirection.UL -> (i+1,j-1)
          | GradDirection.UC -> (i+1,j+0)
          | GradDirection.UR -> (i+1,j+1)
          | GradDirection.CL -> (i,j-1)
          | GradDirection.CR -> (i,j+1)
          | _ -> (i,j)
        let (_i, _j) = (Math.Clamp(I, 1, N-2), Math.Clamp(J,1,N-2))
        let _p = mesh[_i, _j]
        if distance_constraint (I,J) antennas_idx && sides_constraint _p sides N
then
          antennas_idx[k] <- (_i,_j)
          antennas_pos[k] <- mesh[fst antennas_idx[k], snd antennas_idx[k]]
    
```


2. Case Study 2

2.1. Domain

A random domain, NTUA campus was selected from Κτηματολόγιο

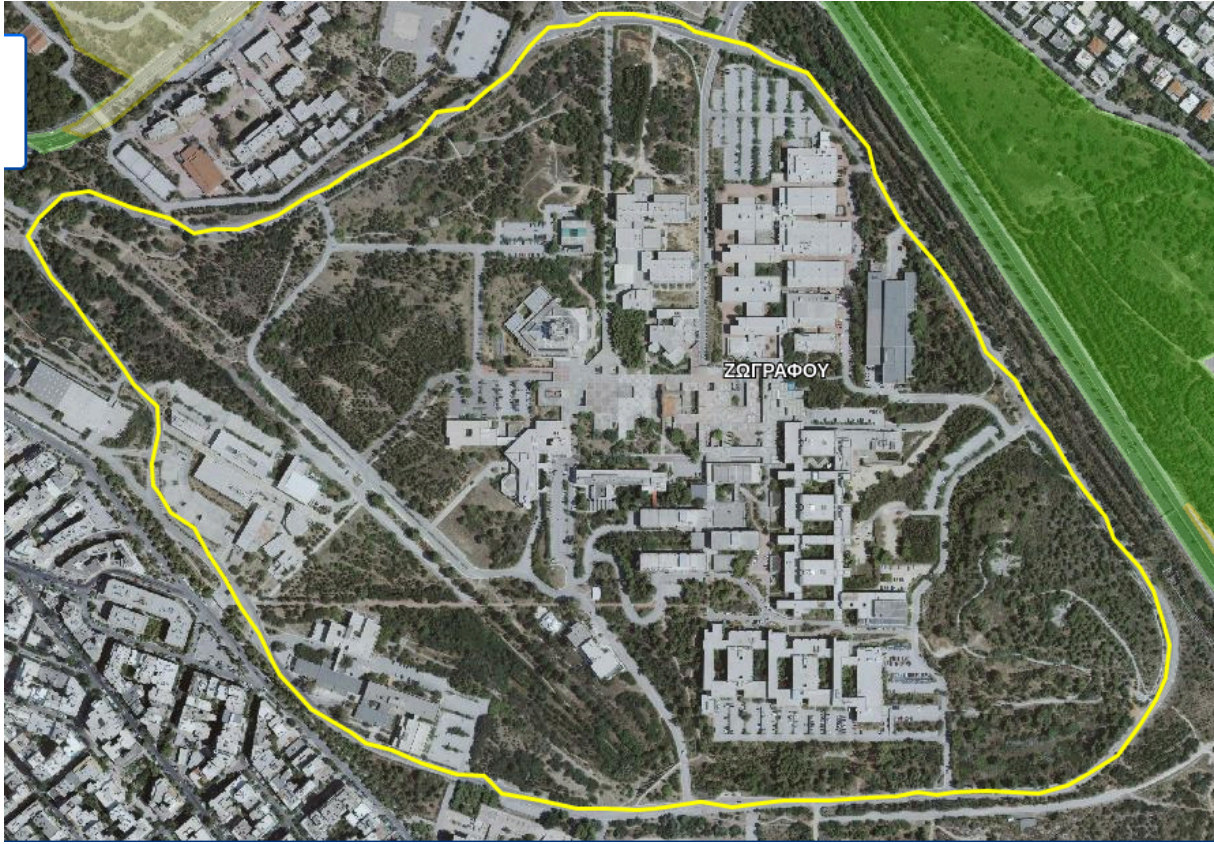


Figure 13: Photograph of the campus, with the designated boundaries

AA	latitude	longitude	AA	latitude	longitude	AA	latitude	longitude
0	481090.588	4202701.507	1	480750.598	4202680.34	2	480520.41	4202747.809
3	480459.556	4202791.466	4	480419.868	4202845.705	5	480411.931	4202881.424
6	480369.597	4202930.372	7	480344.462	4202950.216	8	480318.003	4202983.289
9	480323.295	4203071.924	10	480197.618	4203221.414	11	480193.649	4203242.581
12	480221.43	4203280.946	13	480303.451	4203265.071	14	480384.149	4203239.935
15	480463.524	4203273.008	16	480660.639	4203397.363	17	480680.483	4203438.373
18	480784.994	4203454.248	19	480946.39	4203402.654	20	481008.567	4203311.373
21	481307.547	4202870.841	22	481306.224	4202808.664	23	481248.016	4202737.226
24	481213.62	4202709.445	25	481213.62	4202709.445	26	481089.265	4202700.184
27	481089.265	4202700.184						

Plotting the points with Gnuplot, we get the domain of the problem.²

²The code for the 2nd Case Study is the **structured_mesh.cs** and the **mesh_2_test.fsx**

2.2. Descritization

For the descritization part the algorithmic logic is as follows. We work in two different systems, $\{x, y\} \leftrightarrow \{i, j\}$.

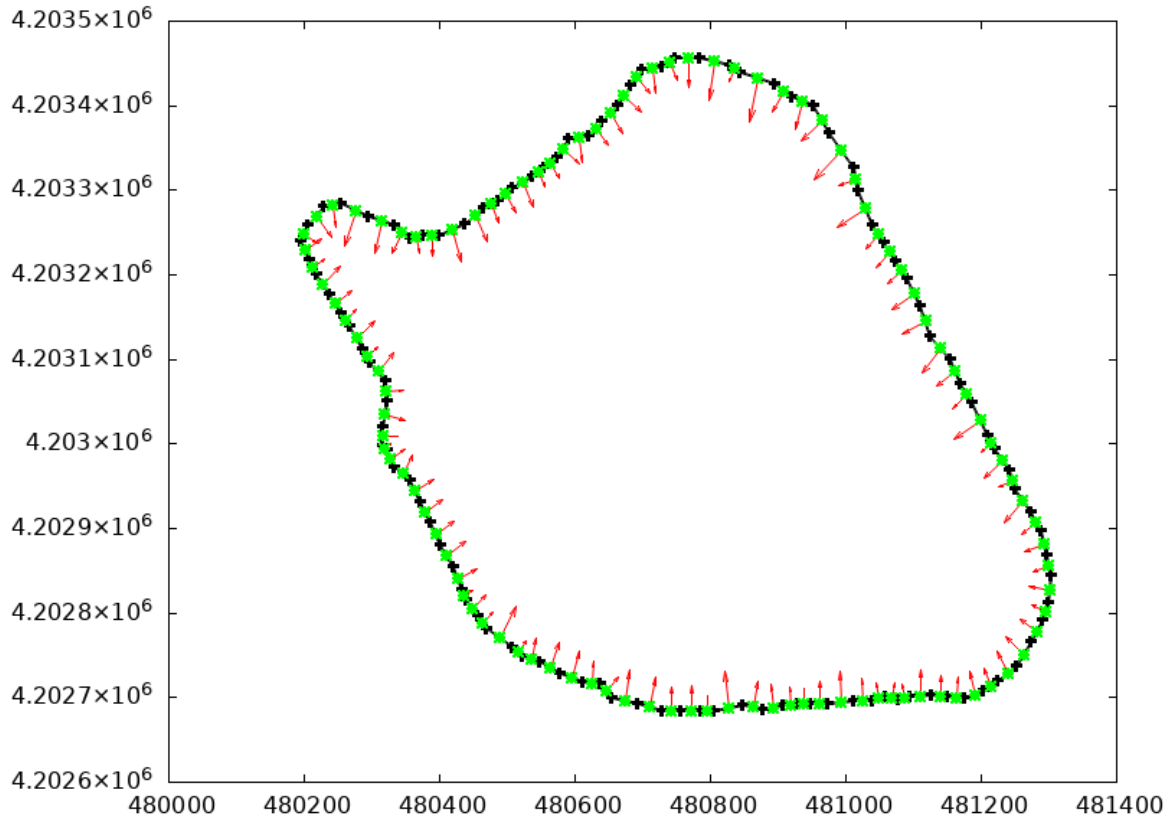


Figure 14: boundary domain, with normals and centers, for each egde

2.2.1. Subdivision

For each edge with compute the normalized coordinates $\{i, j\}$ and check wether the boundary on the grid is continuous. If it is not, the we recursively subdivide to ensure continuity. As an extra utility, a .txt an ASCII file is composed with the normalized coordinates of the boundary points of the domain.

When the continuous boundary in $\{i, j\}$ domain is generated, we iterate on the internal cells of that grid, and fill them as true.

```
for (int i = 0; i < grid.N; i++) {
    int lhs = 0;
    int rhs = grid.N-1;

    while (!grid.stencil_buffer[i*grid.N + lhs]) lhs++; // advance
    while (!grid.stencil_buffer[i*grid.N + rhs]) rhs--; // advance

    while (grid.stencil_buffer[i*grid.N + lhs]) lhs++; // advance
    while (grid.stencil_buffer[i*grid.N + rhs]) rhs--; // advance

    int a = Math.Min(lhs, rhs);
    int b = Math.Max(lhs, rhs);
    bool fill = true;

    switch (MeasureMarchingRow(grid.stencil_buffer, grid.N, i)) {
```



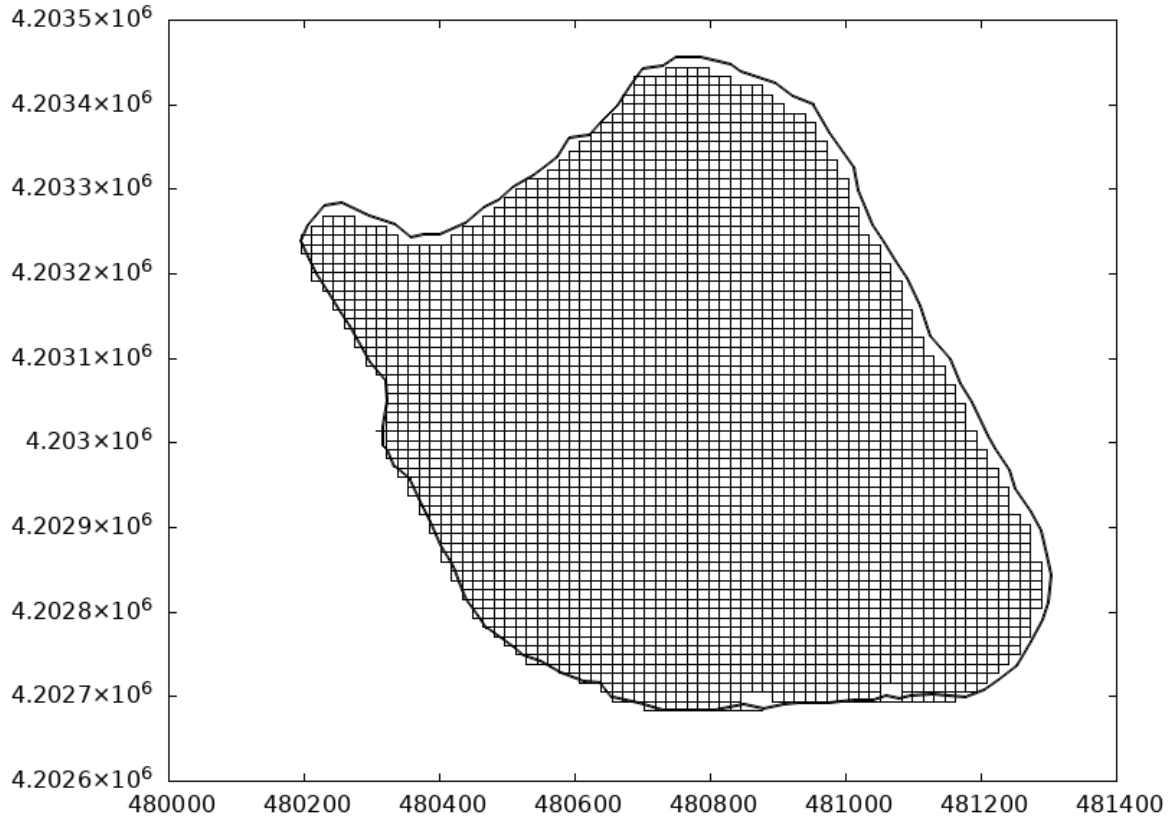
```
case 2:
case 3:
    for (int j = a; j <= b; j++) {
        grid.stencil_buffer[i*grid.N + j] = true;
    }
    break;

case 4:
    for (int j = a; j <= b; j++) {
        if (grid.stencil_buffer[i*grid.N + j]) {
            while (grid.stencil_buffer[i*grid.N + j]) j++; // advance
            j--;
            fill = !fill;
            continue;
        }

        if (fill) {
            grid.stencil_buffer[i*grid.N + j] = true;
        }
    }
    break;
default: break;
}
}
```

This algorithmic logic is akin to RayTracing, MarchingRays algorithms. It has the following short-coming though. When the ray intersects the boundaries it counts the intersections. In case the intersection is 3 (for the selected geometry), then, it hits the edge on the concave spot of the geometry. The current implementation of the algorithm cannot handle that edge case, it requires manual edit of the ASCII file of the geometry. **TODO:** in future further development of the algorithm I will try, in that case to compare the upper and the lower row of that edge case to handle the collision with the edge. I.e. if the both upper and lower row is convex, then the collision with the edge will be ignored. If the upper row is convex and the lower is concave (or vice-versa) then the edge collision will be handled as concave.

The contents of the ASCII file for the geometry, where the edge is manually fixed. Initially it had an extra point in the designated array, the edge collision, which was removed to prevent messing up with counting the collisions with the boundary.



For transforming from $\{x, y\} \rightarrow \{i, j\}$ the following snippet of code is used:

```

/// <summary> transforms to i,j (Stencil system) from x,y (Cartesian system) </
summary>
public static Index ToStencilSystem (int N, Vec2 p, BoundingBox bounds)
{
    double x_min = bounds.v_min.X;
    double x_max = bounds.v_max.X;
    double y_min = bounds.v_min.Y;
    double y_max = bounds.v_max.Y;

    int i = (int)Math.Round((N-1)*(p.Y - y_min) / (y_max - y_min), 0); // normalize
to [0..N-1]
    int j = (int)Math.Round((N-1)*(p.X - x_min) / (x_max - x_min), 0); // normalize
to [0..N-1]

    return new Index{
        I = i,
        J = j,
    };
}
    
```

While for the inverse transform $\{i, j\} \rightarrow \{x, y\}$:

```

/// <summary> transforms to x,y (Cartesian system) from i,j (Stencil system) </
summary>
public static Vec2 ToCartesianSystem (Grid grid, int i, int j)
{
    double x_min = grid.bounds.v_min.X;
    double y_min = grid.bounds.v_min.Y;
    double dx = (grid.bounds.v_max.X - grid.bounds.v_min.X) / (double)grid.N;
    
```

```
double dy = (grid.bounds.v_max.Y - grid.bounds.v_min.Y) / (double)grid.N;

return new Vec2{
    X = (double)j * dx + x_min,
    Y = (double)i * dy + y_min,
};
}
```

2.3. Optimization

For the optimization part of the assignment the following function was used for the measuring the intensity of the signal from the antennas.

$$f_{\text{signal}}(x, y) = A \cdot \exp\left(-\left(\frac{(x - x_0)^2}{2\sigma_x^2} + \frac{(y - y_0)^2}{2\sigma_y^2}\right)\right)$$

A : amplitude

x : x-coordinate of grid node

x_o : x-coordinate of the antenna

y : y-coordinate of grid node

y_o : y-coordinate of the antenna

σ_x : variance on x

σ_y : variance on y

Steepest Descend was utilized for the optimization of the antennas positions.

1. the antennas are assigned random positions inside the domain.
2. the intensity of the signal is calculated for each node of the mesh, as $s_{ij} = \sum_{i=1}^K f_{\text{signal}}(x_j, y_i)$
3. the gradient on each node (of the mesh) is computed.
4. The closest antenna position is displaced w.r.t the signal-intensity on that node and the gradient of that node.
5. after a fixed number of iterations $n_iter_max = 60$, the optimization loop is done and the positions of the antennas during the optimization, are stored into a gif, alongside a serialized text file.

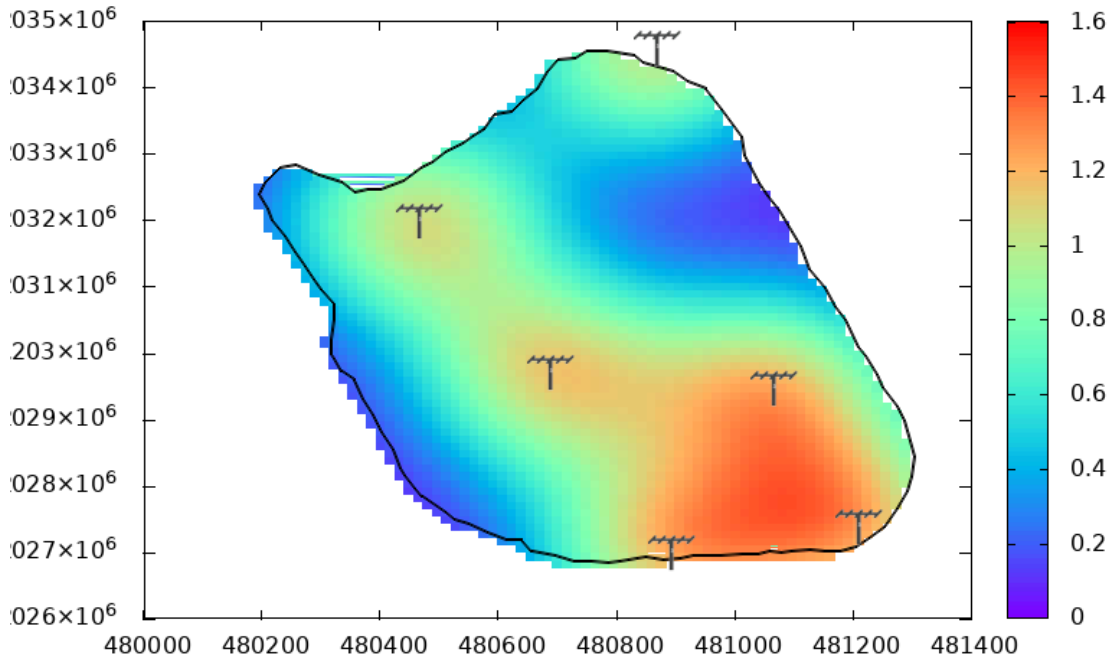


Figure 16: image of the final positions for the antennas and a color gradient for the intensity of the signal. The Optimization algorithm for this image run for just 30 iterations, It is not optimal.

2.4. TODO: Further improvements

Use a totally different datastructure for storing the physical values of the system. Instead of a sparse array, use a Quadtree instead (for 2D problems, or an Octree for 3D). One benefit of that method, is that it has low foot-print on memory. It does not need to cache any coordinate $\{x, y\}$ values, the only essential information for the mesh, is the `stencil_buffer: []bool` and the bounds: `.{v_min:Vec2, v_max:Vec2}` and the size `N` of the square matrix, mesh/grid. Everything else $\{x, y\}$ or $\{i, j\}$ are computed at runtime on the fly, extracted from the aforementioned fields. To get that approach one step further, is to use a Quadtree for the physical properties of the system. That, way a more compressed data structure, will make the algorithm even more memory efficient, as it fits to the design of the grid, so it *should* be a *straight-forward* implementation. With a Quadtree, a single value could be used for many nodes, where the difference is minimal, and more values when the difference is dense.

3. Bibliography

- Lecture Notes
- Numerical Methods in Java Springer (2022)
- Ray Marching_Michael Walczyk
- Learn OpenGL book (stencil buffer - occlusion - depth testing chapters)
- <https://www.youtube.com/@SebastianLague/videos>